Problem Set 3: Topological Autoencoders and Sheaf Neural Networks

Geometric and Topological Methods in Machine Learning
CPSC 6440 01 / AMTH 3220 01 / CPSC 4844 01 / MATH 5220 01 / MATH 7440 01

Assigned: November 3
Due: November 18, 11:59 PM
2025

1 Persistence Diagrams for PDO and Classification

In this section, you will compute persistence diagrams from patient-derived organoid (PDO) CyTOF data and perform supervised classification using vectorized persistence features. You will work on the notebook available at the following link.

1.1 Data and Preprocessing

Use the provided PDO dataset (e.g., `.h5ad`). Preprocess as in Homework 2: filter low-quality cells/genes, normalize (library size), log-transform, and select highly variable genes (HVGs). Embed cells into a low-dimensional space $X \in \mathbb{R}^{n\times d}$ (recommend $d \in \{10, 20, 50\}$ via PCA; you may also compare PHATE/UMAP as a sensitivity check). Each sample (e.g., a PDO, patient, or condition) forms a *point cloud* in $\mathbb{R}^d$.

1.2 Persistent Homology

Persistent Homology and Persistence Images. Persistent homology is a method from topological data analysis (TDA) that captures the shape or connectivity of data across multiple spatial scales. Given a point cloud or metric space, one builds a *filtration*—a sequence of simplicial complexes constructed by gradually increasing a distance or similarity threshold. As the threshold grows, topological features such as connected components, loops, and voids appear and disappear. Each feature's birth and death form a point (b, d) in a *persistence diagram*, where points farther from the diagonal correspond to more significant topological structures.

Since persistence diagrams are sets of points with variable cardinality, they cannot be directly used in most learning algorithms. *Persistence images* provide a fixed-length vector representation by mapping each diagram point (b, d) to a Gaussian bump on a discretized grid in birth–persistence coordinates $(b, p = d - b)$. The resulting heatmap is flattened into a vector, preserving both geometric and topological information in a differentiable, stable form suitable for machine learning.

In the assignment, for each point cloud X_s you are supposed to:

1. Build a Vietoris–Rips filtration on Euclidean distances.
2. Compute 0D and 1D persistence diagrams $\mathcal{D}_s^{(0)}, \mathcal{D}_s^{(1)}$.

1.3 Vectorizing Persistence for Learning

Convert diagrams to fixed-length features using **persistence images** (PI). Map each point $(b,\ d)$ to birth–persistence coordinates $(b,\ p{=}d-b)$, place a Gaussian at (b, p), and integrate over a 32×32 grid:

$$\mathrm{PI}(u, v) \ = \sum_{(b,p)\in\mathcal{D}} w(b, p)\ \exp\Big(-\tfrac{(u-b)^2+(v-p)^2}{2\sigma^2}\Big),$$

where $w(b,p)$ is a stability weight (e.g., $w = p$ or $w = \min\{p, \tau\}$) and $\sigma > 0$ is a bandwidth. Flatten the PI to a vector $\phi_s^{(0)}$. Construct separate vectors for 0D and 1D and concatenate: $\phi_s = [\phi_s^{(0)} \,\|\, \phi_s^{(1)}]$.

1.4 Classification Protocol

Let y_s denote the label for sample s. Train a linear classifier on ϕ_s:

1. Split samples into 5-folds and train a classifier on each fold.
2. Compare **Logistic Regression**, **Linear SVM**, and **RBF SVM**.
3. Metrics: Accuracy, macro-F1; for multi-class, also report confusion matrix.

1.5 Optimal Transport on PI

Represent each PDO or treatment condition as a probability measure over its persistence diagram points:

$$\mu_s = \sum_i w_i \, (b_i,\, d_i - b_i), \qquad w_i = \frac{d_i - b_i}{\sum_j (d_j - b_j)}.$$

Report the 2-Wasserstein distance $W_2(\mu_s, \mu_t)$ between all pairs of treatments using these diagram-based distributions. How do Wasserstein distances between persistence diagrams reflect structural differences between treatment manifolds?

1.6 What to Submit

- A results table: PI(0D+1D) vs. baselines across metrics; include mean±std over folds.
- Ablations: resolution, σ, embedding dimension d, and inclusion/exclusion of 1D.

Question 1.1. *How do 0D vs. 1D features contribute to classification? Provide an ablation isolating each and discuss when loops (1D) add signal.*

Question 1.2. *Compare embeddings for point clouds (PCA vs. PHATE). Which yields more stable diagrams and better downstream accuracy? Relate observed differences to the geometry of each embedding. You can use the code from previous homeworks.*

2 Topology-Preserving Autoencoders (TopoAE) for PDO Classification

This section introduces topology-preserving autoencoders (TopoAE) [4] applied to patient-derived organoid (PDO) single-cell RNA-seq data. The goal is to learn a low-dimensional latent representation that preserves the topological structure of each PDO's cellular manifold and to use these representations for downstream classification.

2.1 Setup and Baseline Autoencoder

Work in the provided Colab notebook: Notebook Link. Load the PDO single-cell dataset (e.g., `.h5ad`) and perform standard preprocessing: filtering, normalization, log-transformation, and selection of highly variable genes. Construct a feature matrix $X \in \mathbb{R}^{n \times d}$ where rows represent cells and columns represent selected genes. Train a baseline autoencoder with mean squared reconstruction loss:

$$L_{\text{recon}} = \|X - \hat{X}\|_2^2.$$

The encoder maps high-dimensional gene expression profiles to a 2D latent representation $Z \in \mathbb{R}^{n \times 2}$, intended to capture the underlying biological manifold of each PDO.

2.2 Topological Loss (0D persistence via MST)

To ensure that the latent space preserves the topological connectivity of the cellular manifold, compute pairwise distance matrices $A_X, A_Z \in \mathbb{R}^{n\times n}$ for input and latent representations. Approximate 0D persistence pairings using edges from the minimum spanning tree (MST), yielding index sets Π_X and Π_Z. Define the topological loss:

$$L_t = \frac{1}{2}\|A_X[\Pi_X] - A_Z[\Pi_X]\|_2^2 + \frac{1}{2}\|A_Z[\Pi_Z] - A_X[\Pi_Z]\|_2^2.$$

The final training objective combines both reconstruction and topological terms:

$$L = L_{\text{recon}} + \lambda L_t,$$

where $\lambda > 0$ balances geometry and topology preservation.

2.3 Tasks

1. **Implement the TopoAE:** Complete TODOs as described in the notebook.
2. **Train TopoAE:** Optimize L for $\lambda \in \{0.0, 0.1, 0.5, 1.0\}$ while keeping architecture and optimizer fixed.
3. **PDO classification:** Use the topology-aware encoder to extract latent embeddings, aggregated them, train a classifier (e.g., logistic regression, linear SVM) to predict point cloud level labels such as treatment type.

2.4 What to Report

- Representative persistence diagrams (0D, 1D) for PDO latent embeddings.
- Comparison of reconstruction loss and topological loss across λ values.
- PDO-level classification accuracy and F1-score using persistence features.
- Visualizations of the latent embeddings colored by PDO or condition.

Question 2.1. *How does the choice of λ affect the geometry of the PDO latent space and the stability of persistence diagrams across samples?*

No

Question 2.2. *Compare classification accuracy using persistence features derived from (i) raw expression PCA, and (ii) TopoAE latent embeddings. Does topological regularization improve separability?*

? ablation

Question 2.3. *Evaluate the contribution of 0D vs. 1D features to PDO classification. Do topological loops (1D persistence) add discriminative power?*

3 Sheaf Neural Networks for Trajectory Prediction

Sheaf neural networks (SNNs) [3, 2, 1] generalize message-passing graph neural networks by assigning a local *vector space*, or *stalk*, to each node and defining how information is *restricted* and compared across edges through linear *restriction maps*. Formally, let $G = (V, E)$ denote a graph with node features $x_i \in \mathbb{R}^d$. For every edge $(i, j) \in E$, define restriction matrices $R_i^{(ij)}, R_j^{(ij)} \in \mathbb{R}^{d\times d}$ that map local stalk features to a shared edge space. The *compatibility* (or *disagreement*) along edge (i, j) measures how inconsistent the adjacent node embeddings are under these restrictions:

$$\text{compat}_{ij} = R_i^{(ij)} x_i - R_j^{(ij)} x_j.$$

These edge-level signals are then aggregated back to nodes:

$$m_i = \sum_{j:(i,j)\in E} \text{compat}_{ij}, \qquad x_i' = x_i + \sigma\big(W m_i + b\big),$$

where σ denotes a nonlinearity (e.g., tanh or ReLU) and W, b are learnable parameters. This residual form allows iterative diffusion of local disagreements across the graph.

Identity vs. Learned Restrictions. In the **identity restriction** case, the restriction maps are fixed to identity transformations, $R_i^{(ij)} = R_j^{(ij)} = I$, giving $\text{compat}_{ij} = x_i - x_j$, which reduces the layer to a residual Laplacian-like propagation. In the **learned restriction** case, $R_i^{(ij)}$ and $R_j^{(ij)}$ are predicted by a small edge MLP and parameterized as near-identity deformations:

$$R_i^{(ij)} = I + \alpha\,\Delta_i^{(ij)}, \qquad R_j^{(ij)} = I + \alpha\,\Delta_j^{(ij)}, \quad \text{with } \alpha > 0 \text{ small.}$$

This enables the network to learn localized geometric transformations while maintaining numerical stability.

Trajectory Prediction Objective. After L layers of sheaf propagation, node representations $x_i^{(L)}$ are projected to scalar *node logits* through a readout MLP. For next-step trajectory prediction on SCoNe-style synthetic datasets, these logits are gathered into aligned neighbor slots for each current node, forming a categorical distribution over possible next nodes. The model is trained using a cross-entropy loss:

$$\mathcal{L}_{\text{CE}} = -\frac{1}{B}\sum_{b=1}^{B} \log \frac{\exp(\ell_{b,\,u^*})}{\sum_{u\in\mathcal{N}(v_b)} \exp(\ell_{b,\,u})},$$

where $\ell_{b,u}$ is the predicted logit for neighbor u of node v_b, and u^* denotes the true next node in the trajectory. We evaluate the model using Top-1 and Top-5 accuracy over the neighbor logits.

3.1 Data and Feature Construction

Use the provided Colab: Notebook Link. The dataset folder contains:

$$B_1 \in \mathbb{R}^{|V|\times|E|}, \quad (B_2 \text{ for visualization}), \quad \text{flows_in} \in \mathbb{R}^{N\times|E|}, \quad \text{targets} \in \{0,1\}^{N\times D},$$

along with `last_nodes`, `target_nodes`, train/val/test masks, and an undirected graph G.

Aligned neighbor slots. Build $N_{\text{idx}} \in \mathbb{Z}^{|V|\times D}$ (neighbors per node, padded with -1) and align dataset slots to your neighbor ordering to obtain $N_{\text{idx_aligned}}$ and a mask.

Node features. For a batch, compute standardized divergence and a last-node indicator:

$$X_0[b, v, :] \;=\; \big(\ \text{zscore}((\text{flows_in}[b]\cdot B_1^{\top})_v)\ ,\ \mathbb{1}\{v = \text{last_node}_b\}\ \big) \in \mathbb{R}^2.$$

3.2 Identity-Restriction SNN

Let $x \in \mathbb{R}^{B\times|V|\times d}$ be node stalk features. For each edge (i, j), define compatibility (identity restrictions) by

$$\text{compat}_{ij} \;=\; x_i - x_j.$$

Scatter edge messages to endpoints, pass through an MLP, add a residual update, and apply LayerNorm. A linear readout gives node logits $\in \mathbb{R}^{B\times|V|}$, which you gather into aligned neighbor slots $\in \mathbb{R}^{B\times D}$. Train with cross-entropy over valid slots.

3.3 Learned-Restriction SNN (near-identity)

Predict per-edge restrictions $(R_i, R_j) \in \mathbb{R}^{d\times d}$ via an MLP over edge attributes, enforcing a near-identity parameterization:

$$R_i \;=\; I + \alpha\,\Delta_i, \qquad R_j \;=\; I + \alpha\,\Delta_j, \quad \alpha > 0 \text{ small.}$$

Use compatibility $\text{compat}_{ij} = R_i x_i - R_j x_j$ and the same scatter/update/readout pipeline.

3.4 Evaluation and Baseline

Report Top-1 and Top-5 accuracy on validation and test splits. Compare against a Markov baseline built from empirical $P(u \mid v)$ on training transitions.

3.5 Visualization

Provide a filled-mesh (from B_2 if available) plot: draw the prefix path in red, the predicted next hop in blue, and the ground-truth next hop in red (thicker). Include examples for both identity and learned variants.

3.6 What to report

- Validation/test Top-1/Top-5 for identity vs. learned restrictions and the Markov baseline.
- Training/validation loss curves.
- 2–3 qualitative visualizations of prefix+next (correct and incorrect cases).

? all look the same

Question 3.1. *Ablate the hidden dimension $d \in \{32, 64, 128\}$ and the number of layers $L \in \{1, 3, 5\}$. How do these choices trade off accuracy and runtime?*

Question 3.2. *Vary the near-identity scale α for learned restrictions. When does learning R_i, R_j help over identity, and when does it overfit?*

4 Submission Instructions

Each student should submit the following files to **Gradescope** by **November 23, 11:59 PM** :

1. A detailed write-up in pdf form, titled `[last name]_[first name]_ps3_report.pdf`
2. A pdf of three of the notebooks with the code and all the outputs including visualizations, titled `[last name]_[first name]_ps3_notebook_1.pdf`, `[last name]_[first name]_ps3_notebook_2.pdf` and `[last name]_[first name]_ps3_notebook_3.pdf`
3. A .ipynb file of the filled in notebooks, titled `[last name]_[first name]_ps3_notebook_1.ipynb`, `[last name]_[first name]_ps3_notebook_2.pdf` and `[last name]_[first name]_ps3_notebook_3.ipynb`

References

[1] F. Barbero, C. Bodnar, H. S. de Ocáriz Borde, M. Bronstein, P. Veličković, and P. Liò. Sheaf neural networks with connection laplacians. In *Topological, Algebraic and Geometric Learning Workshops 2022*, pages 28–36. PMLR, 2022.

[2] C. Bodnar, F. Di Giovanni, B. Chamberlain, P. Lio, and M. Bronstein. Neural sheaf diffusion: A topological perspective on heterophily and oversmoothing in gnns. *Advances in Neural Information Processing Systems*, 35:18527–18541, 2022.

[3] J. Hansen and T. Gebhart. Sheaf neural networks. *arXiv preprint arXiv:2012.06333*, 2020.

[4] M. Moor, M. Horn, B. Rieck, and K. Borgwardt. Topological autoencoders. In *International conference on machine learning*, pages 7045–7054. PMLR, 2020.